Operating Systems

Complete Notes

Comprehensive notes covering all topics

Source: GeeksforGeeks OS Tutorial

Table of Contents

1. Basics of Operating Systems

1.1 Introduction to Operating System

An Operating System (OS) is software that acts as an intermediary between computer hardware and the user. It manages hardware and software resources of a computing device.

What Does an OS Do?

- Manages computer resources such as CPU, memory, and files
- Acts as an interface between user and hardware
- Performs core functions like process, memory, and file management
- Assigns resources (memory, processors, I/O devices) to processes that need them
- The OS is a program that runs at all times on a computer

User Interaction Interfaces

- Command-Line Interface (CLI) — e.g., Bash, PowerShell
- Graphical User Interface (GUI) — e.g., Windows Desktop, macOS Finder

Goals of an Operating System

Primary Goals:

Goal	Description
User Convenience	Provide a user-friendly interface for easy interaction
Program Execution	Facilitate execution of user programs with necessary environment and services
Resource Management	Manage and allocate CPU, memory, disk storage, and I/O devices for fair utilization
Security	Protect the system and user data from unauthorized access

Secondary Goals:

- Efficient Resource Utilization — Maximize performance of CPU, Memory, and I/O devices
- Reliability — Handle errors and exceptions gracefully; modular design for easy debugging

Components of an OS

- Shell — Outermost layer; handles user interaction; interprets input and output
- Kernel — Core component; primary interface between OS and hardware

Common Operating Systems

OS	Developer	Use Cases
Windows	Microsoft	Personal computing, business, gaming
macOS	Apple	Creative industries, personal computing
Linux	Linus Torvalds / Open-source	Servers, development, personal computing
Unix	AT&T Bell Labs	Servers, workstations, research

Applications of an OS

- Platform for running application programs
- Managing input/output units (keyboard, printer, monitor)
- Multitasking — Manages memory, allows multiple programs to run simultaneously
- File & Memory Management — Allocates/deallocates memory for tasks
- Security — Authorization and access control

1.2 Types of Operating Systems

1. Batch OS

Processes jobs in groups (batches) without user interaction. Jobs queued and executed one after another.

- Advantages: Minimal idle time, handles repetitive tasks, improved throughput
- Disadvantages: Inefficient CPU utilization during I/O, high response time, no real-time feedback
- Examples: Insurance claim processing, library book records

2. Multi-Programming OS

Multiple programs reside in memory at the same time. CPU switches between programs when one is waiting for I/O.

- Advantages: Better CPU utilization, improved throughput, efficient resource use
- Disadvantages: Complex design, security issues, high memory requirement

- Examples: Banking systems, railway servers

3. Multi-Tasking / Time-Sharing OS

A type of multiprogramming where each process gets a fixed time slice (quantum). After quantum ends, OS switches to next task (round-robin).

- Advantages: Equal CPU access, reduced software duplication, low CPU idle time
- Disadvantages: Lower reliability, security concerns, communication issues
- Examples: IBM VM/CMS, TSO, Windows Terminal Services

4. Multi-Processing OS

Uses two or more CPUs for execution. Betters the throughput of the system.

- Advantages: Faster processing, high reliability (fault tolerance), supports heavy tasks
- Disadvantages: High cost, complex design, not always efficient
- Examples: UNIX, Linux, macOS

5. Distributed OS

Connects multiple independent computers through a shared communication network. Each has own CPU and memory but works as single unit.

- Advantages: Independent systems, easily scalable, lower processing delays
- Disadvantages: Network dependency, lack of standardization, high cost & complexity
- Examples: LOCUS, MICROS, Amoeba

6. Network OS (NOS)

Runs on a server and manages data, users, security, and applications. Allows shared access to files, printers, resources.

- Advantages: Centralized and stable servers, easy upgrades, remote access
- Disadvantages: High server cost, dependency on server, regular maintenance needed
- Examples: Microsoft Windows Server, UNIX, Linux

7. Real-Time OS (RTOS)

Processes data and responds within strict time limits.

- Advantages: Maximum resource consumption, error-free, best memory allocation
- Disadvantages: Limited tasks, complex algorithms, thread priority issues
- Examples: Scientific experiments, medical imaging, robots

8. Mobile OS

Designed specifically for smartphones and tablets. Manages hardware, software, apps, touch-based interfaces.

- Advantages: User-friendly, extensive app ecosystems, multiple connectivity options
- Disadvantages: Battery life constraints, security risks, fragmentation (Android)
- Examples: Android, iOS, Blackberry

RTOS Sub-types

Type	Description	Example
Hard RTOS	Strict timing; any delay is unacceptable	Airbags, automatic parachutes
Soft RTOS	Timing important but minor delays acceptable	Multimedia, gaming, video streaming

1.3 Kernel in Operating System

The Kernel is the core part of an OS that acts as a bridge between software applications and hardware.

Functions of the Kernel

- Process Management — Scheduling and execution of processes
- Memory Management — Allocation/deallocation, virtual memory, memory protection
- Device Management — Managing I/O devices, providing unified hardware interface
- File System Management — Managing file operations, providing file system interface
- Resource Management — Managing CPU time, disk space, network bandwidth
- Security & Access Control — Enforcing user permissions and authentication
- Inter-Process Communication — Message passing and shared memory

Types of Kernels

Type	Description	Examples
Monolithic	All OS services run in kernel space; fast but less fault-isolation	Unix, Linux, Open VMS
Microkernel	Minimal kernel; most services in user space; better reliability, more overhead	Minix 3, Mach 3.0
Hybrid	Mix of monolithic + microkernel; balance of speed and safety	Windows NT family, macOS/XNU
Nanokernel	Extremely minimal; only basic hardware abstraction	Nemesis, MIT Exokernel
Exokernel	Separates protection from management; apps get direct hardware control	XOK, Aegis

Working of the Kernel

- Kernel is loaded into memory during boot and stays resident
- Operates in kernel mode (privileged), separate from user mode
- Applications make requests via system calls (software interrupts)
- Kernel switches from user mode → kernel mode to handle requests
- Executes operation (file I/O, process creation, memory allocation)
- Returns result to user space
- Performs context switching for multitasking

1.4 System Calls

A system call is a controlled entry point that allows a user program to request a service from the OS kernel.

How System Calls Work

- User program executes a system call instruction (e.g., syscall or int 0x80)
- CPU switches from user mode → kernel mode
- Kernel identifies the system call number and performs the operation
- After completion, kernel switches back to user mode
- Result (success/failure/data) is returned to the program

> *System calls involve a mode switch, not always a context switch. Context switch happens only when the calling process is blocked.*

Types of System Calls

Category	Purpose	Examples
File System	Create, open, read, write, manage files/dirs	open(), read(), write(), close()
Process Control	Create, execute, synchronize, terminate processes	fork(), exec(), wait(), exit()
Memory Mgmt	Allocate, deallocate, manage memory	mmap(), brk(), malloc()
IPC	Communication between processes	pipe(), shmget(), msgget()
Device Mgmt	Request/release devices, read/write	ioctl(), read(), write()

Common Examples

- fopen() in C internally uses open() system call
- Running a program in Linux uses fork() and exec()
- Printing to screen uses write() system call

1.5 System Initialization (Boot Process)

When we turn on a computer, the following sequence occurs:

- 1. Power On — Hardware receives electrical power
- 2. POST (Power-On Self-Test) — BIOS/UEFI checks hardware components
- 3. BIOS/UEFI Initialization — Firmware initializes and configures hardware
- 4. Boot Loader Execution — Loads OS kernel into memory (e.g., GRUB, Windows Boot Manager)
- 5. Kernel Initialization — Sets up memory management, device drivers, system processes
- 6. Init/Systemd — First user-space process starts remaining services
- 7. User Login — System is ready for user interaction

2. Process Scheduling

2.1 Process Introduction

A process is a program in execution. It is an active entity unlike a program, which is a passive entity.

Process Components

- Code Section — The compiled program code
- Data Section — Global and static variables
- Heap — Dynamically allocated memory
- Stack — Temporary data (function parameters, return addresses, local variables)
- Program Counter — Address of the next instruction to execute
- Registers — CPU register contents

2.2 Process Control Block (PCB)

The PCB (also called Task Control Block) is a data structure maintained by the OS for each process. It contains:

Field	Description
Process ID (PID)	Unique identifier for the process
Process State	Current state (New, Ready, Running, Waiting, Terminated)
Program Counter	Address of next instruction to execute
CPU Registers	Contents of all process-centric registers
CPU Scheduling Info	Priority, scheduling queue pointers
Memory Mgmt Info	Page tables, segment tables, base/limit registers
Accounting Info	CPU time used, time limits, process numbers
I/O Status Info	List of I/O devices allocated, open files

2.3 Process States

A process transitions through: New → Ready → Running → Terminated | Running ↔ Waiting ↔ Ready

State	Description
New	Process is being created
Ready	Waiting to be assigned to a processor
Running	Instructions being executed
Waiting (Blocked)	Waiting for I/O completion or signal
Terminated	Process has finished execution

State Transitions

- New → Ready: Process admitted to the ready queue
- Ready → Running: Scheduler dispatches the process
- Running → Waiting: Process needs I/O or event
- Waiting → Ready: I/O or event completed
- Running → Ready: Interrupt occurs (preemption)
- Running → Terminated: Process completes execution

2.4 Process Schedulers

Scheduler	Description	Frequency
Long-Term (Job)	Selects from pool and loads into memory	Infrequent
Short-Term (CPU)	Selects from ready queue and allocates CPU	Very frequent
Medium-Term	Swaps processes in/out of memory (swapping)	Moderate

Dispatcher vs Scheduler

- Scheduler — Decides WHICH process should run next
- Dispatcher — Actually gives control of CPU to the selected process; handles context switching, switching to user mode, jumping to correct location

2.5 CPU Scheduling Algorithms

Key Terminologies

Term	Definition
Arrival Time	Time at which the process arrives in the ready queue
Burst Time	Time required by a process for CPU execution
Completion Time	Time at which the process completes execution
Turn Around Time	Completion Time − Arrival Time
Waiting Time	Turn Around Time − Burst Time
Response Time	Time from submission until first response is produced

Design Criteria for CPU Scheduling

- CPU Utilization — Keep CPU as busy as possible (40-90% in real systems)
- Throughput — Number of processes completed per unit time

- Turn Around Time — Total time from submission to completion
- Waiting Time — Time spent waiting in the ready queue
- Response Time — Time until first response is produced

Preemptive vs Non-Preemptive Scheduling

- Preemptive: Running process can be interrupted and moved to ready state
- Non-Preemptive: Process runs until it terminates or enters waiting state

1. FCFS (First Come, First Serve)

Processes served in order of arrival. Non-preemptive. Simple to implement. Drawback: Convoy effect — short processes wait behind long ones. Large average waiting time.

2. SJF (Shortest Job First)

Process with smallest burst time selected next. Non-preemptive. Gives minimum average waiting time. Drawback: Can cause starvation for long processes. Difficult to predict burst time.

3. SRTF (Shortest Remaining Time First)

Preemptive version of SJF. Running process preempted if new process arrives with shorter remaining time. Optimal avg waiting time. Drawback: High overhead, starvation possible.

4. Round Robin (RR)

Each process gets a fixed time quantum (time slice). After quantum expires, process moves to end of ready queue. Preemptive. Fair — every process gets equal CPU time. Best for time-sharing systems. Too large quantum → FCFS; too small → too many context switches.

5. Priority Scheduling

Each process assigned a priority; highest priority executes first. Can be preemptive or non-preemptive. Drawback: Starvation — low-priority may never execute. Solution: Aging.

6. HRRN (Highest Response Ratio Next)

Non-preemptive. Response Ratio = (Waiting Time + Burst Time) / Burst Time. Highest ratio selected. Prevents starvation by considering waiting time.

7. Multilevel Queue Scheduling (MLQ)

Ready queue divided into multiple separate queues (e.g., foreground, background). Each queue has own scheduling algorithm. Drawback: Starvation possible.

8. Multilevel Feedback Queue (MLFQ)

Processes can move between queues based on behavior. Most flexible. CPU-bound → lower-priority queues; I/O-bound → higher-priority queues.

Comparison of CPU Scheduling Algorithms

Algorithm	Basis	Preemptive	Starvation	Best For
FCFS	Arrival order	No	No	Simple batch systems
SJF	Shortest burst	No	Yes	Min avg waiting time
SRTF	Shortest remaining	Yes	Yes	Short job preference
RR	Time quantum	Yes	No	Time-sharing systems
Priority	Priority value	Both	Yes	Priority-based
MLQ	Queue priority	No	Yes	Categorized workloads
MLFQ	Queue + feedback	No	No	General purpose

Starvation and Aging

- Starvation: Process waits indefinitely because higher-priority processes keep arriving
- Aging: Gradually increasing priority of processes that wait for a long time; prevents starvation

3. Process Synchronization

3.1 Inter-Process Communication (IPC)

IPC is a mechanism that allows processes to communicate and synchronize their actions.

Method	Description
Shared Memory	Processes share a region of memory for data exchange
Message Passing	Processes communicate by sending/receiving messages
Pipes	Unidirectional data flow between processes
Sockets	Communication over network or same machine
Signals	Asynchronous notifications sent to processes

Types of Processes

- Independent Process — Execution does not affect other processes
- Cooperative Process — Can affect or be affected by other processes

3.2 Process Synchronization

Process Synchronization is the coordination of multiple cooperating processes to ensure controlled access to shared resources, preventing race conditions.

Why Synchronization is Needed

- Prevent race conditions
- Ensure mutual exclusion
- Coordinate processes (e.g., producer-consumer)
- Prevent deadlock
- Ensure safe, ordered communication
- Prevent starvation (fairness)

Types of Process Synchronization

- Competitive — Processes compete for shared resource accessibility
- Cooperative — Processes affect each other's execution

Problems Without Proper Synchronization

- Inconsistency — Conflicting changes to shared data
- Loss of Data — Overwrites before completion
- Deadlock — Processes stuck waiting for each other

3.3 Race Condition

A race condition occurs when the outcome of execution depends on the particular order in which multiple processes/threads access shared data. Occurs inside the critical section. Results in unpredictable, incorrect outcomes. Solution: Ensure mutual exclusion in critical sections.

3.4 Critical Section Problem

The critical section is a code segment where shared variables are accessed/modified. Only one process should be in its critical section at a time.

Property	Description
Mutual Exclusion	Only one process can be in the critical section at a time
Progress	If no process is in critical section, selection of next cannot be postponed indefinitely
Bounded Waiting	Limit on how many times others can enter after a process has made a request

Structure of Critical Section Code

Entry Section → Request permission to enter | Critical Section → Access shared resources | Exit Section → Release critical section | Remainder Section → Non-critical code

3.5 Solutions to the Critical Section Problem

Software Solutions:

- Peterson's Algorithm — Works for 2 processes; uses flag[] (intent) and turn (whose turn); satisfies all 3 requirements. Limitation: only 2 processes, may not work on modern processors due to instruction reordering.
- Dekker's Algorithm — First known correct solution for 2 processes; uses flag[] and turn variables.
- Bakery Algorithm (Lamport's) — Works for N processes; each process takes a number (like bakery ticket); smallest number enters critical section; ensures FCFS ordering.

Hardware Solutions:

- Test-and-Set Lock (TSL) — Atomic instruction that tests and sets a lock
- Compare-and-Swap (CAS) — Atomically compares and swaps values
- Disable Interrupts — Prevent context switching during critical section (only for single processor)

3.6 Semaphores

A Semaphore is a synchronization tool (integer variable) accessed through two atomic operations:

Type	Description
Binary Semaphore (Mutex)	Value can be 0 or 1; used for mutual exclusion
Counting Semaphore	Any non-negative integer; controls access to resource with multiple instances

Semaphore Operations

- wait(S) / P(S) / down(S): while(S<=0) wait; S−−
- signal(S) / V(S) / up(S): S++

Busy Waiting Problem

When a process waits in a loop checking the semaphore, it wastes CPU cycles. Solution: Use blocking semaphores — process is put to sleep and woken up when resource is available.

3.7 Mutex vs Semaphore

Feature	Mutex	Semaphore
Type	Locking mechanism	Signaling mechanism
Values	Locked or Unlocked (0 or 1)	Any non-negative integer
Ownership	Only the owner can unlock	Any process can signal
Purpose	Mutual exclusion	Controlling multiple resource instances
Usage	Protecting critical section	Producer-consumer, reader-writer

3.8 Monitors

A Monitor is a high-level synchronization construct that encapsulates shared data, operations, and synchronization into a single module. Only one process can be active within the monitor at a time. Uses condition variables with wait() and signal() operations. Simpler and less error-prone than semaphores.

Condition Variables in Monitors

- wait() — Process is suspended and placed in a queue
- signal() — Wakes up one waiting process
- broadcast() — Wakes up all waiting processes

3.9 Priority Inversion

Priority Inversion occurs when a high-priority process is indirectly preempted by a lower-priority process.

Scenario: Task L (low) holds a resource. Task H (high) needs that resource and waits. Task M (medium) preempts L. Now H is effectively blocked by M.

Solutions:

- Priority Inheritance Protocol — Temporarily raise L's priority to H's level
- Priority Ceiling Protocol — Assign a ceiling priority to each resource

3.10 Classical IPC Problems

1. Producer-Consumer (Bounded Buffer)

Producer generates data, places in buffer. Consumer takes data. Synchronization prevents: writing to full buffer, reading from empty buffer, simultaneous access.

2. Readers-Writers Problem

Multiple readers can read simultaneously. Only one writer at a time. No reader reads while writer writes. Variations: Reader-priority, Writer-priority.

3. Dining Philosophers Problem

5 philosophers, 5 forks; each needs 2 forks to eat. Must avoid deadlock and starvation. Solutions: Resource ordering, semaphores, monitors.

4. Sleeping Barber Problem

Barber sleeps when no customers. Customers wait if barber busy (limited chairs). If all chairs full, customer leaves. Synchronization using semaphores.

4. Deadlock

4.1 Introduction to Deadlock

Deadlock is a state where two or more processes are stuck forever because each is waiting for a resource held by another.

How Deadlock Occurs

A process uses resources in this order: Request → Use → Release. Example: P1 holds R1, requests R2. P2 holds R2, requests R1. Neither can proceed → Deadlock.

4.2 Necessary Conditions (all 4 must hold simultaneously)

Condition	Description
1. Mutual Exclusion	Only one process can use a resource at a time (non-sharable)
2. Hold and Wait	Process holds at least one resource while waiting for others
3. No Preemption	Resource cannot be forcibly taken from a process
4. Circular Wait	Processes form a circular chain, each waiting for resource held by next

4.3 Deadlock Handling Methods

Method 1: Deadlock Prevention

Ensure at least one necessary condition cannot hold:

Condition	Prevention Strategy
Mutual Exclusion	Use sharable resources where possible (e.g., read-only files)
Hold and Wait	Require process to request all resources at once before starting
No Preemption	If can't get all resources, release what it holds
Circular Wait	Impose ordering on resources; request in increasing order

Method 2: Deadlock Avoidance (Banker's Algorithm)

Each process declares maximum resources needed. System maintains Available, Max, Allocation, and Need matrices. Before granting a request, check if resulting state is safe.

Safe State: A state where there exists a sequence (safe sequence) in which all processes can complete.

Algorithm Steps: 1) Process requests resources. 2) System pretends to allocate and checks for safe state. 3) If safe → grant; if unsafe → wait.

Matrix	Description
Available	Number of available instances of each resource type
Max	Maximum demand of each process
Allocation	Currently allocated to each process
Need	Max − Allocation (remaining need)

Method 3: Detection & Recovery

Detection:

- Use Resource Allocation Graph (RAG) for single-instance resources — cycle → deadlock
- Use algorithm similar to Banker's for multi-instance resources

Recovery:

- Process Termination: Abort all deadlocked processes, or abort one at a time until resolved
- Resource Preemption: Select a victim, rollback to safe state, ensure no starvation

Method 4: Deadlock Ignorance (Ostrich Algorithm)

Simply ignore the problem. Used in most general-purpose OS (Windows, Linux). Assumption: Deadlocks occur rarely and cost of prevention is too high.

4.4 Resource Allocation Graph (RAG)

A directed graph: Process nodes (circles), Resource nodes (rectangles with dots). Request edge: Process→Resource (wants). Assignment edge: Resource→Process (allocated).

> *Single-instance resources: Cycle = Deadlock. Multi-instance: Cycle is necessary but NOT sufficient.*

4.5 Starvation, Livelock & Deadlock

Concept	Description
Starvation	Process waits indefinitely; resources always given to others
Livelock	Processes continuously change state but make no progress (like two people in a hallway)
Deadlock	Processes completely stuck, not executing at all

5. Multithreading

5.1 What is a Thread?

A thread is a single sequence of execution within a process. Also called a lightweight process. Each thread belongs to exactly one process.

- Threads share: code section, data section, OS resources (open files, signals)
- Each thread has its own: Thread ID, Program Counter, Register Set, Stack

Thread Components

Component	Description
Stack Space	Stores local variables, function calls, return addresses
Register Set	Holds temporary data and intermediate results
Program Counter	Tracks current instruction being executed

5.2 Benefits of Multithreading

- Improved Performance — Threads run in parallel
- Increased Responsiveness — One thread responds while another is busy
- Concurrency — Multiple operations happen simultaneously
- Simpler Communication — Threads share memory; no need for IPC
- Efficient Context Switching — Faster than process switching (same address space)
- Better Multiprocessor Utilization — Threads can run on different cores
- Resource Sharing — Threads share code, data, and files
- Higher Throughput — More jobs completed per unit time

5.3 Types of Threads

User-Level Threads (ULTs)

Managed entirely in user space using a thread library. Kernel is unaware. Fast context switching (no system calls). Limitation: If one thread blocks, entire process blocked. Cannot fully utilize multiprocessor systems. Scheduling done by application.

Kernel-Level Threads (KLTs)

Managed directly by OS kernel. Each thread has entry in kernel's thread table. Kernel schedules independently → true parallel execution. If one blocks, kernel runs another from same process. Slower context switching (user↔kernel mode). More complex.

Comparison: ULT vs KLT

Feature	User-Level Threads	Kernel-Level Threads
Management	Thread library	OS Kernel
Kernel Awareness	No	Yes
Context Switch	Fast	Slower
Blocking Call	Blocks entire process	Only blocks that thread
Multiprocessor	No	Yes
Complexity	Simple	Complex

5.4 Multithreading Models

Model	Description
Many-to-One	Many user threads → one kernel thread. Fast but no parallelism. If one blocks, all block.
One-to-One	Each user thread → one kernel thread. True parallelism. Overhead of creating kernel threads.
Many-to-Many	Many user → many kernel (<=user). Best flexibility. Kernel can schedule on multiple processors.

5.5 Process vs Thread

Feature	Process	Thread
Memory	Separate memory space	Shared memory space
Independence	Independent	Not independent
Creation	Heavyweight, expensive	Lightweight, cheaper
Communication	Through IPC	Direct memory access
Context Switch	Slower	Faster
Resources	Own resources	Shares process resources

5.6 Process-based vs Thread-based Multitasking

Feature	Process-based	Thread-based
Unit	Processes	Threads
Address Space	Separate	Shared
Switching Cost	High	Low
Communication	Complex (IPC)	Simple (shared memory)

6. Memory Management

6.1 Introduction

Memory Management is the process of controlling and organizing a computer's memory by allocating blocks to different executing programs to improve overall system performance.

Key Functions

- Supports multiple processes simultaneously in memory
- Protects processes from unauthorized access
- Enables swapping and virtual memory efficiently
- Tracks which parts of memory are in use and which are free

Memory Hierarchy

Registers → Cache → Main Memory (RAM) → Secondary Storage (Disk)

Fastest/Smallest/Most Expensive ↔ Slowest/Largest/Cheapest

Logical vs Physical Address

Type	Description
Logical Address	Generated by the CPU; also called virtual address
Physical Address	Actual address in main memory
MMU	Memory Management Unit — Hardware that maps logical to physical addresses

6.2 Memory Allocation Techniques

Swapping

Processes temporarily moved between main memory and secondary storage. Allows multiple processes to run efficiently. Lower-priority processes swapped out for higher-priority. Transfer time depends on amount of data moved.

Contiguous Memory Allocation

Each process allocated a single continuous block of memory.

Single Contiguous Allocation:

Memory divided into two parts: OS + one user process. No multiprogramming possible. Simple but poor utilization.

Fixed (Static) Partitioning:

Memory divided into fixed number of partitions with fixed sizes. Each holds one process. Leads to internal fragmentation (allocated > process size). Tracked using partition table.

Variable (Dynamic) Partitioning:

Partitions created dynamically based on process size. Reduces internal fragmentation. Suffers from external fragmentation (scattered free blocks).

Memory Placement Algorithms

Algorithm	Strategy	Characteristics
First Fit	Allocate first block large enough	Fast; may leave small fragments
Best Fit	Allocate smallest block that fits	Minimizes waste; slower search
Worst Fit	Allocate largest block	Leaves largest remaining fragment
Next Fit	First Fit from last allocation point	Avoids clustering at beginning

Buddy System

Memory allocated in powers of 2. When block freed, merges with its 'buddy' if also free. Reduces external fragmentation.

6.3 Non-Contiguous Allocation

Process divided into smaller parts stored in different, non-adjacent memory locations.

Paging

Divides process into fixed-size pages. Physical memory into fixed-size frames (same size). Page table maps pages to frames. No external fragmentation (possible internal fragmentation in last page). Address: Logical = Page number + Page offset.

Page Table Entries:

Field	Purpose
Frame Number	Physical frame where page is stored
Valid/Invalid Bit	Whether page is in memory
Protection Bits	Read/Write/Execute permissions
Reference Bit	Whether page was recently accessed
Dirty (Modified) Bit	Whether page has been modified

Segmentation

Divides process into logical segments of variable size (code, data, stack). Each segment has name and length. Uses segment table with base address and limit. Supports logical view. Drawback: External fragmentation.

Segmentation with Paging

Combines logical segmentation with paging. Each segment divided into pages. Reduces fragmentation while maintaining logical structure.

6.4 Virtual Memory

Virtual Memory allows execution of processes not completely in memory. Lets a program run even if larger than physical RAM.

Key Concepts

- Uses disk as extension of main memory
- Only needed portion of program needs to be in memory
- More programs can run simultaneously
- Provides illusion of large contiguous memory to each process

Demand Paging

Loads only pages a process actually needs. Page loaded only on page fault (not in memory). Reduces unnecessary I/O. Uses lazy swapper (pager) — never swaps in unless needed.

Page Fault Handling

- 1. Process tries to access a page not in memory
- 2. Page fault trap is generated
- 3. OS checks if the reference is valid
- 4. If valid, OS finds a free frame
- 5. Page is loaded from disk into the free frame
- 6. Page table is updated
- 7. Instruction is restarted

Swap Space

Area on secondary storage (disk) used to hold pages swapped out of memory. Acts as overflow for main memory.

6.5 Page Replacement Algorithms

When memory is full and a new page needs to be loaded, the OS must decide which page to remove.

Algorithm	Strategy	Notes
FIFO	Replace oldest page	Simple. Belady's Anomaly possible
Optimal	Replace page not used for longest future time	Min page faults. Not practical (needs future knowledge)
LRU	Replace least recently used page	Good approx of Optimal. No Belady's Anomaly
LFU	Replace lowest access count	May not reflect recent usage
MFU	Replace highest access count	Least-used may be just brought in
Second Chance (Clock)	Modified FIFO with reference bit	If ref=0 replace; if 1 set to 0, move on. Circular queue

Belady's Anomaly: For FIFO, increasing frames can INCREASE page faults. LRU and Optimal are immune. Stack-based algorithms are immune.

6.6 Thrashing

Thrashing occurs when the system spends more time swapping pages than executing processes.

Causes

- Too many processes in memory; each has fewer frames than needed; high page fault rate

Effects

- CPU utilization drops drastically; system appears to 'freeze'; throughput decreases

Solutions

- Working Set Model — Track set of pages a process is actively using; allocate enough frames
- Page Fault Frequency (PFF) — Monitor page fault rate; adjust frame allocation
- Suspend/Swap processes — Reduce degree of multiprogramming
- Local Replacement — A process can only replace its own pages

6.7 Kernel Memory Allocation

Buddy System

Divides memory into blocks of powers of 2. Adjacent free blocks (buddies) merged. Used for kernel memory allocation.

Slab Allocation

Pre-allocates memory for specific kernel objects (e.g., PCBs, inodes). Slabs grouped into caches. Reduces fragmentation and allocation overhead. Used in Linux kernel.

6.8 Additional Memory Concepts

Overlays

Legacy technique for running programs larger than available memory. Programmer manually divides program into overlays. Only needed overlay loaded at a time. Replaced by virtual memory.

Memory Interleaving

Distributes memory addresses across multiple memory modules. Allows simultaneous access. Increases memory bandwidth.

OS-based Virtualization

Creates virtual machines sharing same hardware. Each VM has own virtual memory space. Hypervisor manages memory allocation between VMs.

7. Disk Management

7.1 File Systems

A file system controls how data is stored and retrieved from storage devices.

File System	Used By	Features
FAT32	USB drives, older Windows	Simple, widely compatible, 4GB file limit
NTFS	Windows	Journaling, permissions, large files
ext4	Linux	Journaling, large volume support
APFS	macOS	Optimized for SSDs, encryption
HFS+	Older macOS	Hierarchical, journaling

Unix File System

Tree-structured directory hierarchy starting from root (/). Everything treated as a file (regular files, directories, devices). Uses inodes to store file metadata.

7.2 File Directory & Path Names

Types of Paths

- Absolute Path — Full path from root (e.g., /home/user/file.txt)
- Relative Path — Path from current directory (e.g., ./file.txt)

Directory Structures

Structure	Description
Single-Level	All files in one directory; naming conflicts
Two-Level	Separate directory for each user
Tree-Structured	Hierarchical; subdirectories allowed
Acyclic Graph	Allows sharing of files/directories (links)
General Graph	Allows cycles; requires garbage collection

7.3 File Allocation Methods

Method	Description	Advantages	Disadvantages
Contiguous	Files in consecutive blocks	Fast sequential & direct access	Ext. fragmentation, needs pre-known size
Linked	Each block has pointer to next	No ext. fragmentation	Slow random access, pointer overhead
Indexed	Index block has all pointers	Direct access, no ext. frag.	Index block overhead

7.4 File Access Methods

Method	Description
Sequential Access	Data read/written one record after another (like a tape)
Direct (Random) Access	Data accessed directly by block number
Indexed Sequential	Uses an index to speed up sequential access

7.5 Secondary Memory (HDD)

HDD Components

- Platters — Circular disks coated with magnetic material
- Tracks — Concentric circles on platter surface
- Sectors — Divisions of a track (smallest addressable unit)
- Cylinders — Set of tracks at same position on all platters
- Read/Write Head — Reads/writes data on platter surface
- Spindle — Rotates platters at high speed (5400–15000 RPM)

Key Time Components

Component	Description
Seek Time	Time to move disk arm to the correct track
Rotational Latency	Time for desired sector to rotate under the head
Transfer Time	Time to actually read/write data
Disk Access Time	Seek Time + Rotational Latency + Transfer Time

7.6 Disk Scheduling Algorithms

Disk scheduling determines the order in which disk I/O requests are processed. Goals: Minimize seek time, maximize throughput, minimize latency, ensure fairness.

1. FCFS

Requests served in arrival order. Simple and fair. Drawback: Does not optimize seek time; may cause large head movements.

2. SSTF (Shortest Seek Time First)

Selects request closest to current head position. Reduces avg response time. Drawback: Can cause starvation of distant requests.

3. SCAN (Elevator Algorithm)

Disk arm moves in one direction servicing requests, then reverses. High throughput, low variance. Drawback: Edge requests wait longer.

4. C-SCAN (Circular SCAN)

Like SCAN but when arm reaches end, jumps back to beginning without servicing. More uniform wait time.

5. LOOK

Like SCAN but arm only goes to last request in each direction (not disk end). Reduces unnecessary movement.

6. C-LOOK (Circular LOOK)

Like C-SCAN but goes only to last request. Reduced head movement compared to C-SCAN.

Other Algorithms

- RSS (Random Scheduling) — Used in simulation and analysis
- LIFO — Newest job serviced first; maximizes locality but can cause starvation
- N-STEP SCAN — Buffer N requests; service them before accepting new ones; eliminates starvation
- F-SCAN — Uses two sub-queues; prevents arm stickiness and ensures fairness

Comparison of Disk Scheduling Algorithms

Algorithm	Seek Opt.	Starvation	Fairness	Complexity
FCFS	None	No	High	Low
SSTF	High	Yes	Low	Medium
SCAN	Good	No	Medium	Medium
C-SCAN	Good	No	High	Medium
LOOK	Better	No	Medium	Medium
C-LOOK	Better	No	High	Medium

7.7 Spooling and Buffering

Spooling (Simultaneous Peripheral Operations On-Line)

Data temporarily held in a buffer (spool) on disk while being transferred between devices. Allows overlap of I/O with processing. Example: Print spooling — documents queued on disk before printing.

Buffering

A buffer is a temporary memory area holding data during transfer between two devices. Compensates for speed differences. Types: Single buffer, Double buffer, Circular buffer.

Feature	Spooling	Buffering
Storage	Disk	Main memory
Scope	Multiple jobs overlap	One job at a time
Efficiency	Better for I/O-bound tasks	Speed mismatch compensation

7.8 Free Space Management

Method	Description
Bit Vector (Bitmap)	Each block = 1 bit (0=free, 1=occupied)
Linked List	Free blocks linked together
Grouping	First free block stores addresses of n free blocks
Counting	Store address of first free block + count of contiguous free blocks

8. Interrupts

8.1 What is an Interrupt?

An interrupt is a signal generated by hardware or software that alerts the processor to an event requiring immediate attention. It causes the CPU to temporarily

stop current execution and respond to a high-priority request.

- IRQ (Interrupt Request Line) — Hardware line used to generate interrupts
- ISR (Interrupt Service Routine) — Software routine executed to handle an interrupt
- Processor completes current instruction before servicing the interrupt
- Interrupt Latency — Delay between receiving interrupt and starting ISR execution

8.2 Types of Interrupts

Software Interrupts

Generated by software or system instructions (not hardware). Also called traps and exceptions. Occur when system calls are made or exceptions happen (e.g., division by zero). Examples: fork() system call, division by zero exception.

Hardware Interrupts

Generated by hardware devices connected to the Interrupt Request Line. Device closes its associated switch to request an interrupt. Value of INTR = logical OR of requests from individual devices.

Type	Description
Maskable Interrupt	Can be selectively enabled/disabled via interrupt mask register
Non-Maskable (NMI)	Cannot be ignored by CPU; used for critical events (hardware failures)
Spurious Interrupt	Hardware interrupt with no identifiable source (phantom/ghost interrupts)

8.3 Interrupt Handling Mechanism

- 1. Interrupt raised (I/O or system interrupt)
- 2. Current state (registers, PC) saved to conserve process state
- 3. Interrupt identified through Interrupt Vector Table
- 4. Control shifts to interrupt handler (kernel space)
- 5. ISR performs specific tasks to manage the interrupt
- 6. Saved state restored from previous session
- 7. Control returns to interrupted process; normal execution continues

8.4 Managing Multiple Devices

Method	Description
Polling	First device with IRQ bit set is serviced first. Easy but wastes time interrogating all devices.
Vectored Interrupts	Device sends special code (interrupt vector) identifying itself directly; fast identification.
Interrupt Nesting	Devices organized in priority; higher-priority recognized, lower-priority not.

8.5 Interrupt Priority Schemes

Scheme	Description
Fixed Priority	Each interrupt has predetermined priority; highest handled first
Dynamic Priority	Priority changes based on system conditions; for real-time tasks
Vectored Scheme	Each interrupt has specific memory address (vector); CPU jumps there
Priority Masking	Lower-priority interrupts temporarily disabled
Round-Robin	Cyclic order; fair handling when all have same priority

8.6 Triggering Methods

Method	Description
Level-Triggered	Signal must be held at active level; device maintains until CPU acknowledges
Edge-Triggered	Caused by level change (rising/falling edge); pulse driven then released

8.7 Benefits of Interrupts

- Real-time Responsiveness — Respond promptly to external events
- Efficient Resource Usage — No busy-waiting; processor idle until event, saving power
- Multitasking & Concurrency — Handle multiple tasks by switching
- Improved Throughput — Overlap computation with I/O operations

9. Quick Reference: Key Formulas & Comparisons

Formula	Description
TAT = Completion Time − Arrival Time	Turn Around Time
Waiting Time = TAT − Burst Time	Time spent waiting in ready queue
CPU Util. = (Busy / Total) × 100%	CPU utilization percentage
Throughput = Processes / Total Time	Processes completed per unit time
Disk Access = Seek + Rot. Latency + Transfer	Total disk access time
EAT = (1−p)×ma + p×(page fault time)	p=fault rate, ma=memory access time
Page Table Size = Pages × Entry Size	Memory needed for page table
Num Pages = Virtual Addr Space / Page Size	Total pages in virtual memory
Num Frames = Physical Memory / Frame Size	Total frames in physical memory

Key Differences

Process vs Thread

Process	Thread
Heavyweight	Lightweight
Separate memory	Shared memory
Slow context switch	Fast context switch
Independent	Dependent

Paging vs Segmentation

Paging	Segmentation
Fixed-size pages	Variable-size segments
No external fragmentation	External fragmentation possible
Internal fragmentation possible	No internal fragmentation
Physical division	Logical division

Preemptive vs Non-Preemptive Scheduling

Preemptive	Non-Preemptive
Process can be interrupted	Runs until completion/wait
Better response time	Simpler implementation
Higher overhead	Lower overhead
Used in time-sharing	Used in batch systems

Deadlock vs Starvation vs Livelock

Deadlock	Starvation	Livelock
Circular wait	Indefinite wait	Continuous state change
All processes stuck	One process affected	Processes keep responding
No progress at all	Some progress by others	No actual progress

10. Interview Questions

Q1. What is a process and process table?
Ans: A process is an instance of a program in execution. The OS maintains a process table to track the state of all processes, listing resources used and current state.

Q2. What are the different states of a process?
Ans: Running (executing), Ready (waiting for CPU), Waiting (waiting for I/O). Also: New (being created), Terminated (finished).

Q3. What is a Thread?
Ans: A thread is a single sequence stream within a process (lightweight process). Threads share code, data, and files but have their own stack, registers, and program counter.

Q4. What is Thrashing?
Ans: Thrashing occurs when the system spends more time processing page faults than executing. CPU utilization drops as page fault rate increases.

Q5. What is virtual memory?
Ans: Virtual memory creates an illusion of large contiguous address space by using disk to extend RAM. Pages loaded on demand.

Q6. What is a kernel?
Ans: The central component of an OS managing hardware, memory, and CPU time. Acts as bridge between applications and hardware. Types: Monolithic, Micro, Hybrid.

Q7. What are the different scheduling algorithms?
Ans: FCFS, SJF, SRTF, Priority, Round Robin, MLQ, MLFQ, HRRN.

Q8. What is demand paging?
Ans: Only required pages loaded into RAM on page fault, not the entire process. Reduces unnecessary memory usage.

Q9. What is the difference between process and thread?
Ans: Processes: independent, separate memory, IPC for communication, heavyweight. Threads: share memory, direct communication, lightweight, faster context switch.

Q10. What is context switching?
Ans: Saving the state (PCB) of current process and loading saved state of new process so CPU can switch between them. Involves saving PC, registers, memory info.

Q11. What is a zombie process?
Ans: A process that finished execution but still has an entry in the process table. Parent must read its exit status to remove it.

Q12. What are orphan processes?
Ans: Processes whose parent terminated without waiting for the child. The init process (PID 1) adopts them.

Q13. What is PCB?
Ans: Process Control Block stores: PID, state, program counter, registers, scheduling info, memory info, I/O status. Process table = array of PCBs.

Q14. Preemptive vs non-preemptive scheduling?
Ans: Preemptive: CPU can be taken from running process (better response, higher overhead). Non-preemptive: runs until completion/I/O (simpler, lower overhead).

Q15. What are starvation and aging?
Ans: Starvation: process never gets resources. Aging: gradually increasing priority of waiting processes to prevent starvation.

Q16. What is the critical section problem?
Ans: When multiple processes access shared code/data, only one in critical section at a time. Must satisfy: mutual exclusion, progress, bounded waiting.

Q17. What are the necessary conditions for deadlock?
Ans: All 4 simultaneously: Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait. Removing any one prevents deadlock.

Q18. What is Banker's Algorithm?
Ans: Deadlock avoidance: simulates allocation of max resources, checks safe state before granting. Uses Available, Max, Allocation, Need matrices.

Q19. How to recover from a deadlock?
Ans: Process termination (abort all or one-at-a-time) or Resource preemption (select victim, rollback, ensure no starvation).

Q20. What is paging?
Ans: Memory management: process split into fixed-size pages, memory into frames. Page table maps pages to frames. Eliminates external fragmentation.

Q21. What is fragmentation?
Ans: Internal: allocated > needed (wasted within partition). External: scattered free blocks can't satisfy requests. Solutions: compaction, paging, segmentation.

Q22. Paging vs Segmentation?
Ans: Paging: fixed-size, physical division, no external frag, possible internal frag. Segmentation: variable-size, logical division, external frag possible.

Q23. What are semaphores?
Ans: Integer sync variables with atomic wait() and signal(). Binary (0/1) for mutex; counting for multiple resources. Busy-wait problem solved by blocking.

Q24. What are classic synchronization problems?
Ans: Bounded-Buffer (Producer-Consumer), Readers-Writers, Dining Philosophers, Sleeping Barber. Solved using semaphores or monitors.

Q25. What is Belady's Anomaly?
Ans: In FIFO page replacement, increasing frames can paradoxically increase page faults. Does NOT occur with LRU or Optimal.

Q26. What are interrupts?
Ans: Signals alerting CPU to handle an event. Hardware interrupts (maskable/non-maskable) from devices; Software interrupts (traps/exceptions) from instructions. Handled by ISR via Interrupt Vector Table.

Q27. What is spooling?
Ans: Simultaneous Peripheral Operations On-Line. Data buffered on disk before sending to peripheral, allowing CPU to continue working.

Q28. What is a dispatcher?
Ans: Module giving process CPU control after scheduler selects it. Handles context switching, user mode switch, jumping to correct location.

Q29. What are the goals of CPU scheduling?
Ans: Max CPU utilization, max throughput, min turnaround time, min waiting time, min response time, fair CPU allocation.

Q30. What is a safe state in deadlock avoidance?
Ans: A state where at least one sequence (safe sequence) exists in which all processes can complete without deadlock.